CS285 Assignment 3: Q-Learning and Actor-Critic Algorithms

Xiaolei Chu (Student ID: 3038525739)

March 10, 2026

1 Deep Q-Learning

1.1 Basic Q-Learning (Section 2.4)

We trained a DQN agent on CartPole-v1. As shown in Figure 1, the agent reaches the maximum return of 500 during training, confirming a correct DQN implementation.

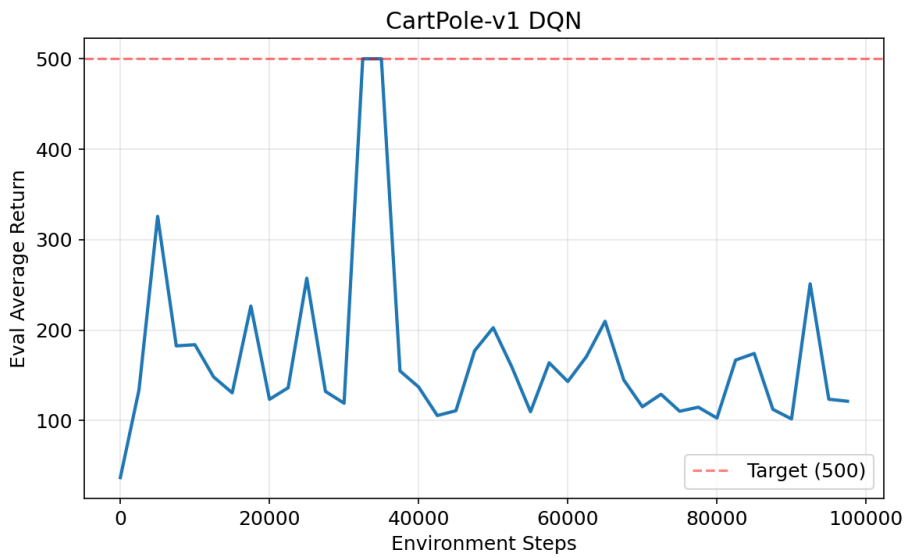

Figure 1: CartPole-v1 DQN: Eval average return over environment steps. The agent reaches the maximum possible return of 500.

1.2 Double Q-Learning (Section 2.5)

1.2.1 LunarLander-v2

We trained a Double DQN agent on LunarLander-v2. As shown in Figure 2, the agent achieves a return above 200 during training (peak: 275.2), meeting the expected threshold.

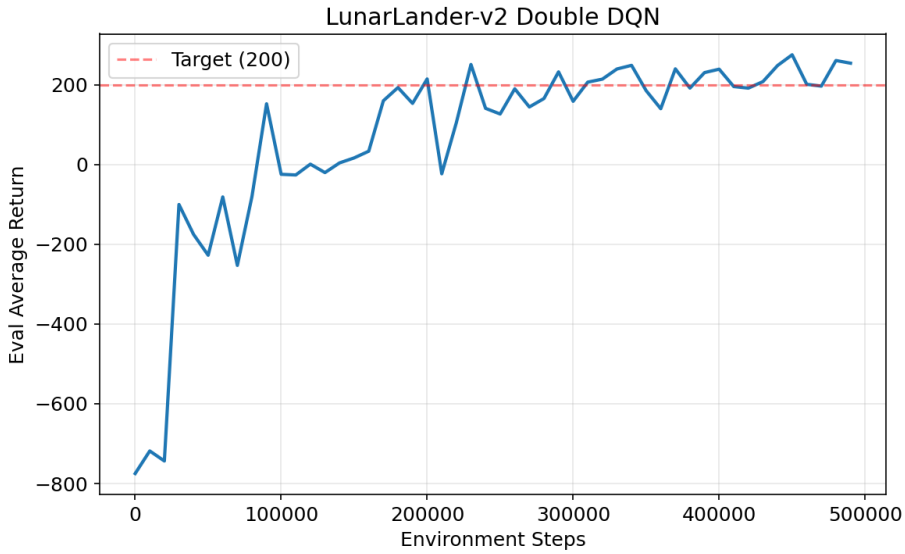

Figure 2: LunarLander-v2 Double DQN: Eval average return over environment steps.

1.2.2 MsPacman

We trained DQN with double-Q learning on MsPacman. Figure 3 shows both the training return and eval return over training steps. The agent achieves a peak eval return of 3315, well above the 1500 target.

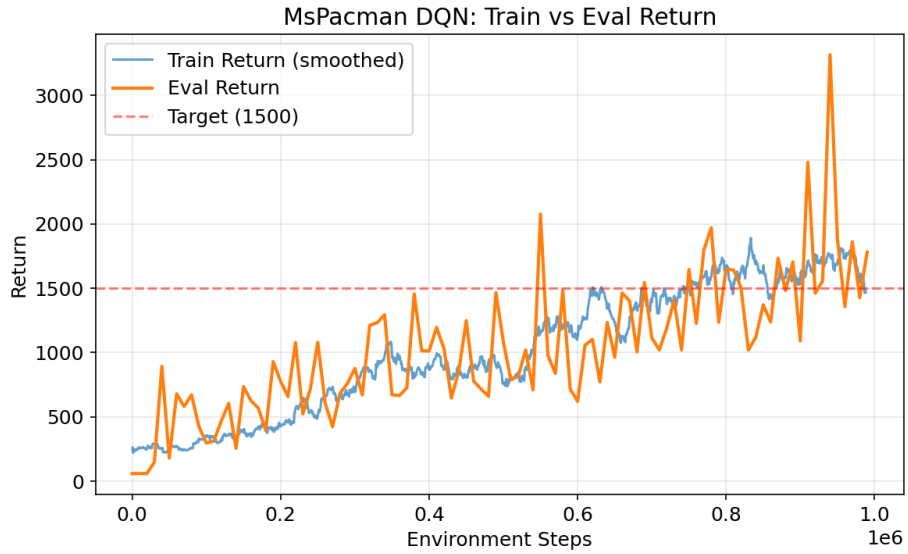

Figure 3: MsPacman DQN: Training return (smoothed) and eval return plotted together.

Explaining the difference between train and eval returns: Early in training, the training return is significantly lower than the eval return. This is because during training, the agent uses an ϵ-greedy exploration strategy with a high ϵ value (close to 1.0), meaning most actions are taken randomly, which leads to poor performance. In contrast, during evaluation, the agent acts greedily

(or with a very small ϵ), using the learned Q-values to select the best actions. As training progresses and ϵ decays, the training return gradually approaches the eval return because the agent explores less and exploits its learned policy more. Additionally, eval episodes are averaged over multiple rollouts, which can lead to higher and more stable returns compared to individual training episodes.

1.3 Hyperparameter Experiment (Section 2.6)

We chose to experiment with the **learning rate** on LunarLander-v2, testing four values: 1×10^{-2}, 1×10^{-3} (default), 5×10^{-4}, and 1×10^{-4}.

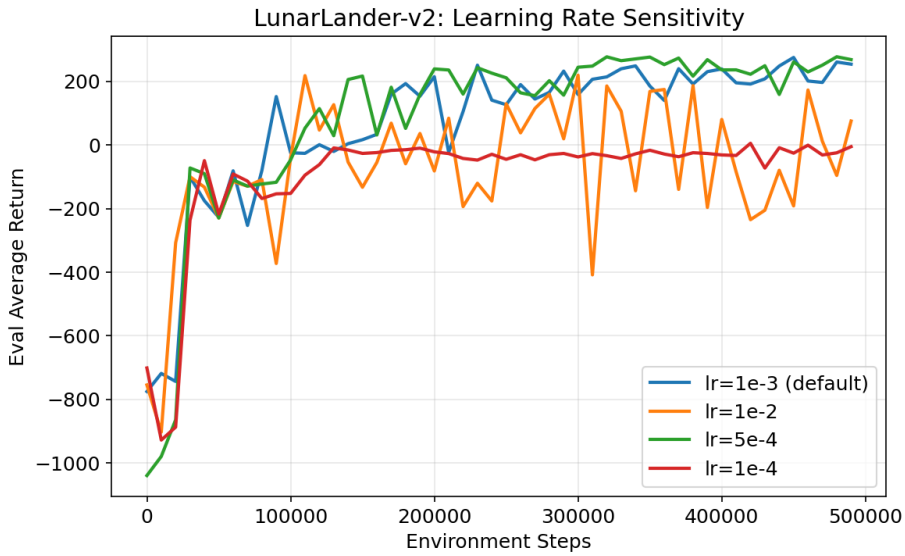

Figure 4: LunarLander-v2: Learning rate sensitivity. We chose the learning rate because it directly controls the magnitude of gradient updates and is one of the most critical hyperparameters in deep RL. Too large a learning rate (lr=1×10^{-2}) causes instability, while too small (lr=1×10^{-4}) leads to extremely slow learning that fails to converge within the training budget. The default lr=1×10^{-3} and lr=5×10^{-4} both achieve good performance, with lr=5×10^{-4} performing slightly better (peak 277.4 vs 275.2).

2 Soft Actor-Critic (SAC)

2.1 Actor Update with Reparametrization (Section 3.4)

We trained SAC on HalfCheetah-v4 with a fixed temperature of $\beta = 0.1$. As shown in Figure 5, the agent achieves a peak eval return of 9254, well above the 6000 target.

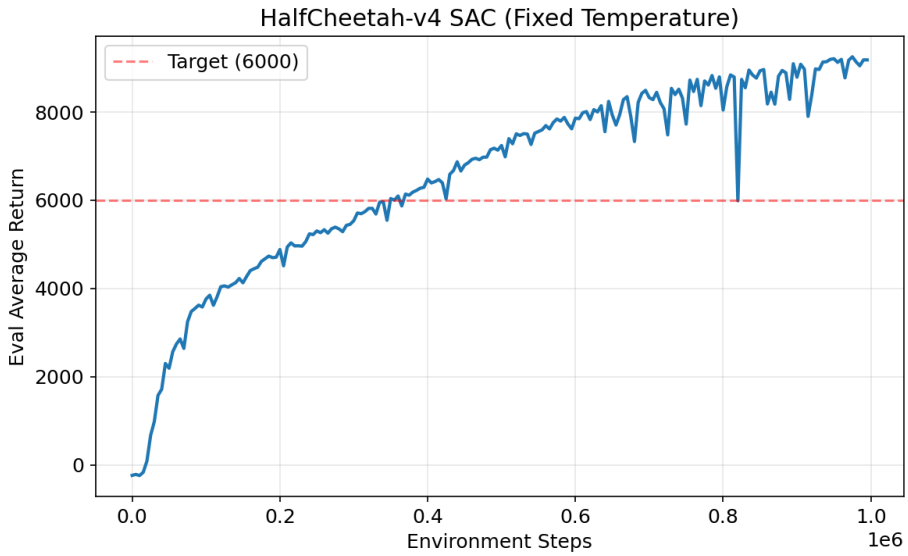

Figure 5: HalfCheetah-v4 SAC with fixed temperature: Eval return over environment steps.

2.2 Automatic Temperature Tuning (Section 3.5)

Figure 6 compares fixed-temperature and auto-tuned SAC on HalfCheetah-v4, and shows the evolution of the learned temperature α over training.

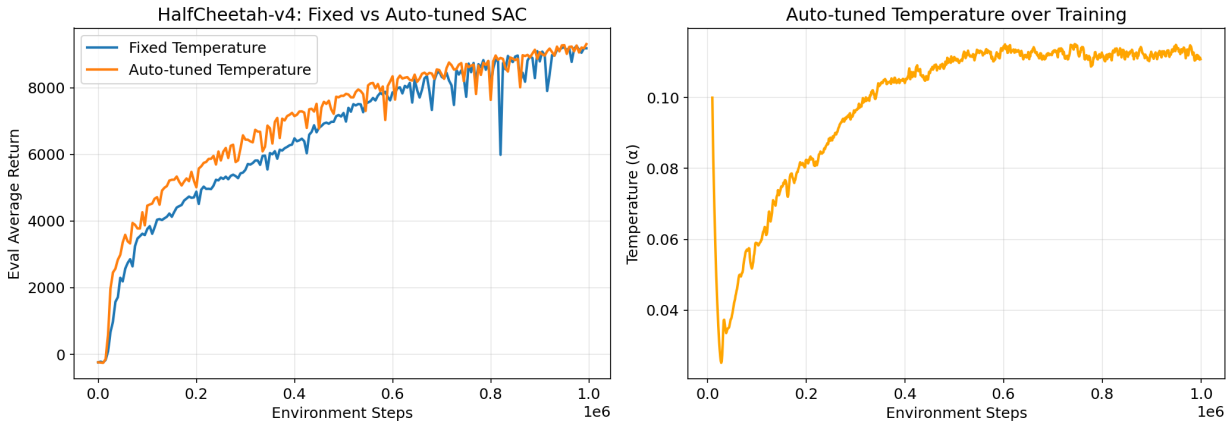

Figure 6: Left: Eval return comparison between fixed ($\beta = 0.1$) and auto-tuned SAC on HalfCheetah-v4. Right: Learned temperature α over training for the auto-tuned run.

Does auto-tuning improve or achieve comparable performance to the fixed temperature? Auto-tuning achieves comparable performance to the fixed temperature setting. Both reach peak returns around 9250–9300, with the auto-tuned version showing slightly smoother learning in the early stages. Since the fixed temperature ($\beta = 0.1$) is already well-tuned for HalfCheetah, the main advantage of auto-tuning is eliminating the need for manual temperature selection.

How does the temperature evolve during training? The temperature initially drops sharply from 0.1 to approximately 0.025 in the first $\sim$100k steps, then gradually increases back to

around 0.11 where it stabilizes for the remainder of training.

Why might the temperature change in this particular way for HalfCheetah? The initial decrease occurs because early in training, the policy is nearly random and already has high entropy, so the algorithm reduces the entropy bonus to allow the critic to learn more accurate Q-values without excessive noise. As training progresses and the policy becomes more deterministic (converging toward exploitation), the temperature increases to maintain sufficient exploration and prevent premature convergence. The eventual stabilization around 0.11 (close to the manually-tuned value of 0.1) indicates that the dual gradient descent has found a balance between exploration and exploitation that satisfies the target entropy constraint, which for HalfCheetah's 6-dimensional action space is $\mathcal{H}_{\text{target}} = -6$.

2.3 Stabilizing Target Values (Section 3.6)

Figure 7 compares single-Q and clipped double-Q on Hopper-v4.

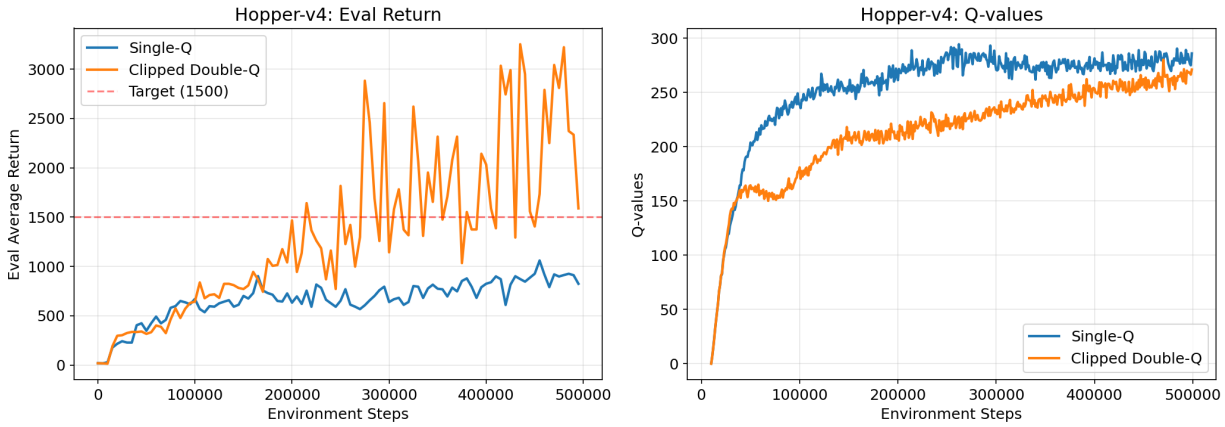

Figure 7: Hopper-v4: Left: Eval return for single-Q vs clipped double-Q. Right: Q-values over training.

Which one works best? Clipped double-Q significantly outperforms single-Q. The clipped double-Q agent achieves a peak return of 3252, while the single-Q agent only reaches 1060.

Discussion of overestimation bias: The Q-values plot clearly illustrates the overestimation problem. The single-Q agent's Q-values grow substantially larger than those of the clipped double-Q agent, despite achieving much lower actual returns. This is a classic symptom of overestimation bias: the single critic bootstraps from its own potentially overestimated values, creating a feedback loop that drives Q-values to unrealistically high levels. These inflated Q-values mislead the actor into taking suboptimal actions, resulting in poor performance.

The clipped double-Q approach mitigates this by using $\min(Q_{\phi'_A}(s', a'), Q_{\phi'_B}(s', a'))$ for the target value computation. Since two independently trained critics are unlikely to overestimate the same state-action pairs simultaneously, taking the minimum provides a more conservative and accurate value estimate. This leads to more stable training and substantially better policy performance, as evidenced by the much higher eval returns despite lower Q-value estimates.